BagKeeper: the “Who-is-who” Pickup Check

Code walkthrough for a task write-up. Commit ec2b91e. Python / Flask / SQLite + a little JavaScript.

1. What the feature does

BagKeeper lets a shopper leave bags with a mall runner. The risk: a stranger pretends to be the runner. This feature makes the runner and the shopper prove who they are **before the bag is sealed and tagged**:

• **Step 1 (runner shows a code).** Both phones show the same rotating phrase, e.g. *Amber Fox 42*. It changes every 60 seconds. The runner holds up their screen.

• **Step 2 (shopper approves).** The shopper compares it with their own phone and taps **“It matches”**. Only then is the shopper's 4-digit PIN revealed.

• **Step 3 (runner enters the PIN).** The shopper tells the runner the PIN. 3 wrong tries lock the step and alert an admin.

• **Step 4 (runner seals and tags).** The seal/tag form is refused by the server until steps 2 and 3 are done.

Why a rotating phrase: it is computed from a per-order secret and the current minute (HMAC), so it cannot be guessed or screenshotted and reused later. A stranger without the app cannot show the right phrase.

2. Files touched (only the relevant parts are shown)

• **bagkeeper/db.py**: new order columns (PIN, secret, verified_at, pin_ok_at, fail count, lock)

• **bagkeeper/services.py**: all the rules (phrase, confirm, PIN, lock, seal guard)

• **bagkeeper/web.py**: the two form routes (shopper confirm, runner PIN)

• **bagkeeper/api.py**: JSON endpoints the pages poll (phrase, status)

• **bagkeeper/static/verify.js**: refreshes the phrase, auto-advances the runner's screen

• **bagkeeper/templates/order.html**: the shopper card and the runner's Step 1 / Step 2 cards

3. Data model: db.py

Columns added to `orders` by the lightweight migration in `init_db()`:

```
"orders": [("pickup_pin", "TEXT"),            # shopper's 4-digit PIN for the runner
           ("verify_secret", "TEXT"),         # seeds the rotating two-word phrase
           ("verified_at", "TEXT"),           # shopper confirmed the runner's phrase
           ("pin_ok_at", "TEXT"),             # runner entered the right PIN
           ("pin_fails", "INTEGER NOT NULL DEFAULT 0"),
           ("pin_locked", "INTEGER NOT NULL DEFAULT 0"),
           ...
```

The PIN and secret are generated when the order is created (`services.create_order`):

```
f"{secrets.randbelow(10000):04d}", secrets.token_hex(16)   # -> pickup_pin, verify_secret
```

4. The rules: services.py (lines 811-937)

Read it in this order: **phrase_for** (builds the code) → **can_see_phrase** (who may see it) → **confirm_runner** (shopper approves) → **enter_pin** (runner unlocks) → **verification_needed** (the gate used when sealing).

```
# ---------- who is who: runner <-> shopper verification ----------
```

```
# Pickup: the shopper compares the runner's rotating phrase with their own screen and taps
# "It matches"; only then does the shopper reveal a 4-digit PIN, which the runner types in to
# unlock sealing. Delivery: the same phrase check reveals the handover code. A shopper can also
# report "this isn't my runner", which puts the job on hold for the admin.
PIN_MAX_TRIES = 3
PHRASE_WINDOW_S = 60
ADJECTIVES = (
    ... (62 adjectives)
)
ANIMALS = (
    ... (62 animals)
)


def _utc_text():
    from datetime import datetime, timezone
    return datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S")


def _secret(order):
    if order["verify_secret"]:
        return order["verify_secret"]
    s = secrets.token_hex(16)            # orders created before this feature
    get_db().execute("UPDATE orders SET verify_secret=? WHERE id=? AND verify_secret IS NULL",
                     (s, order["id"]))
    get_db().commit()
    return get_db().execute("SELECT verify_secret FROM orders WHERE id=?", (order["id"],)).fetchone()[0]


def phrase_for(order, now=None):
    """The two-word-plus-number phrase both phones show right now, and seconds until it changes.
    Derived from a per-order secret and the current minute, so it can't be guessed or reused."""
    import hashlib
    import hmac
    import time
    now = time.time() if now is None else now
    window = int(now // PHRASE_WINDOW_S)
    digest = hmac.new(_secret(order).encode(), f"{order['id']}:{window}".encode(),
                      hashlib.sha256).digest()
    adj = ADJECTIVES[int.from_bytes(digest[0:2], "big") % len(ADJECTIVES)]
    animal = ANIMALS[int.from_bytes(digest[2:4], "big") % len(ANIMALS)]
    number = int.from_bytes(digest[4:6], "big") % 100
    return f"{adj} {animal} {number:02d}", int(PHRASE_WINDOW_S - (now % PHRASE_WINDOW_S))


def can_see_phrase(order, user):
    """Only the shopper who owns the order and the runner assigned to it."""
    if order is None or order["status"] not in ("assigned", "out_for_delivery") or order["safety_flag"]:
        return False
    return (user["role"] == "customer" and order["customer_id"] == user["id"]) or \
           (user["role"] == "runner" and order["runner_id"] == user["id"])


def _check_not_flagged(o):
    if o["safety_flag"]:
        raise RuleError("This job is on hold after a safety report. Wait for the admin.")


def verification_needed(o):
    """Pickup can't start until the shopper confirmed the phrase and the runner entered the PIN.
    Orders from before this feature (no PIN) are exempt."""
    return o["pickup_pin"] is not None and not (o["verified_at"] and o["pin_ok_at"])


def confirm_runner(order_id, customer_id):
    """Shopper taps 'It matches' (pickup) or confirms the runner again at delivery."""
    o = get_order(order_id)
    if o is None or o["customer_id"] != customer_id:
        raise RuleError("Not your order.")
    _check_not_flagged(o)
    if o["runner_id"] is None or o["status"] not in ("assigned", "out_for_delivery"):
        raise RuleError("Your runner isn't on the way yet.")
    badge = o["runner_badge"] or "-"
```

```
    if o["status"] == "assigned":
        if o["verified_at"]:
            return
        _set(order_id, verified_at=_utc_text())
        log(order_id, customer_id, "runner_verified", f"shopper confirmed the phrase (runner badge {badge})")
    else:
        if o["delivery_verified_at"]:
            return
        _set(order_id, delivery_verified_at=_utc_text())
        log(order_id, customer_id, "delivery_runner_verified", f"shopper confirmed the phrase (runner badge {badge})")
    get_db().commit()


def enter_pin(order_id, runner_id, pin):
    """Runner types the shopper's PIN. Three wrong tries lock the step until an admin overrides."""
    import hmac
    o = get_order(order_id)
    if o is None or o["runner_id"] != runner_id or o["status"] != "assigned":
        raise RuleError("This pickup isn't yours.")
    _check_not_flagged(o)
    if o["pin_ok_at"] or o["pickup_pin"] is None:
        return
    if not o["verified_at"]:
        raise RuleError("Wait until the shopper confirms your phrase.")
    if o["pin_locked"]:
        raise RuleError("Locked after 3 wrong PINs. Call the admin.")
    pin = (pin or "").strip()
    if not hmac.compare_digest(pin.encode(), o["pickup_pin"].encode()):
        fails = o["pin_fails"] + 1
        locked = fails >= PIN_MAX_TRIES
        _set(order_id, pin_fails=fails, pin_locked=1 if locked else 0)
        log(order_id, runner_id, "pin_failed", f"wrong PIN, attempt {fails} of {PIN_MAX_TRIES}")
        if locked:
            log(order_id, runner_id, "pin_locked", "3 wrong PINs - an admin must review and override")
        get_db().commit()
        raise RuleError("Locked after 3 wrong PINs. The admin has been alerted." if locked
                        else f"Wrong PIN. {PIN_MAX_TRIES - fails} tries left.")
    _set(order_id, pin_ok_at=_utc_text())
    log(order_id, runner_id, "pin_ok", "shopper PIN accepted")
    get_db().commit()
```

5. The gate on sealing and tagging: services.record_pickup

This is the line that makes the check mandatory. Even if someone posts directly to the server, sealing is refused until the shopper has approved and the PIN was entered.

```
def record_pickup(order_id, runner_id, seal_no, seal_method, bag_photo, sealed_photo):
    o = get_order(order_id)
    if o["status"] != "assigned" or o["runner_id"] != runner_id:
        raise RuleError("This pickup isn't assigned to you.")
    _check_not_flagged(o)
    if verification_needed(o):
        raise RuleError("Verify the shopper first: they confirm your phrase, then you enter their PIN.")
    seal_no = seal_no.strip().upper()
```

6. Routes: web.py and api.py

```
# ---------- who is who: runner <-> shopper verification ----------
@bp.post("/order/<int:order_id>/verify/confirm")
@login_required("customer")
def verify_confirm(order_id):
    _order_or_404(order_id)
    return _act(order_id, lambda: svc.confirm_runner(order_id, g.user["id"]))


@bp.post("/order/<int:order_id>/verify/pin")
@login_required("runner")
def verify_pin(order_id):
    return _act(order_id, lambda: svc.enter_pin(order_id, g.user["id"], request.form.get("pin", "")))

@bp.get("/orders/<int:order_id>/verify/phrase")
```

```
@login_required("customer", "runner")
def verify_phrase(order_id):
    o = svc.get_order(order_id)
    if o is None:
        return jsonify(error="not found"), 404
    if not svc.can_see_phrase(o, g.user):
        return jsonify(error="not available"), 403
    phrase, left = svc.phrase_for(o)
    resp = jsonify(phrase=phrase, seconds_left=left, runner=(o["runner"] or "").split(" ")[0],
                   badge=o["runner_badge"])
    resp.headers["Cache-Control"] = "no-store"
    return resp
```

7. Browser side: static/verify.js

Every element marked `data-phrase` fetches the phrase and re-fetches when the minute rolls over. The runner's Step 1 card (`data-wait-verify`) polls the status and reloads into Step 2 the moment the shopper taps “It matches”.

```
(function () {
  document.querySelectorAll("[data-phrase]").forEach(function (box) {
    var id = box.dataset.order;
    var left = box.parentNode.querySelector("[data-phrase-left]");
    var timer = null, tick = null, remaining = 0;

    function show() {
      if (left) left.textContent = remaining > 0 ? "Changes in " + remaining + " s. If it just changed, look again." : "";
    }
    function load() {
      fetch("/api/orders/" + id + "/verify/phrase", { cache: "no-store" })
        .then(function (r) { return r.ok ? r.json() : null; })
        .then(function (j) {
          if (!j) { box.textContent = "-"; if (left) left.textContent = "No phrase right now."; return; }
          box.textContent = j.phrase;
          remaining = j.seconds_left;
          show();
          clearTimeout(timer);
          timer = setTimeout(load, Math.max(2, remaining + 1) * 1000);
        })
        .catch(function () { timer = setTimeout(load, 5000); });
    }
    tick = setInterval(function () { if (remaining > 0) { remaining -= 1; show(); } }, 1000);
    load();
  });

  document.querySelectorAll("[data-wait-verify]").forEach(function (box) {
    var id = box.dataset.order;
    var poll = setInterval(function () {
      fetch("/api/orders/" + id + "/verify/status", { cache: "no-store" })
        .then(function (r) { return r.ok ? r.json() : null; })
        .then(function (j) {
          if (j && (j.shopper_confirmed || j.flagged)) { clearInterval(poll); location.reload(); }
        }).catch(function () {});
    }, 3000);
  });
```

8. Screens: templates/order.html

Shopper: sees the phrase and the “It matches” button; after confirming, the PIN appears.

```
    <div class="card">
      <h2 style="margin-top:0">Check your runner</h2>
      <div class="row"><b>{{ o.runner.split(' ')[0] }}</b><span class="badge">Badge {{ o.runner_badge }}</span></div>
      {% if not o.verified_at %}
        <p class="muted">When your runner arrives: <b>1.</b> Ask them to show their screen. <b>2.</b> Compare the phrase on th
        <div class="code phrase" data-phrase data-order="{{ o.id }}">{{ phrase }}</div>
        <p class="muted" data-phrase-left style="text-align:center"></p>
        <form method="post" action="{{ url_for('web.verify_confirm', order_id=o.id) }}"><button class="btn">It matches</button
      {% else %}
        <p><span class="badge ok">Runner verified</span></p>
        <h3 style="margin:8px 0">Your PIN</h3>
```

```
        <div class="code">{{ o.pickup_pin }}</div>
        <p class="muted">Tell this PIN to your runner so they can start. Never share it with anyone else.</p>
      {% endif %}
```

Runner: Step 1 shows the phrase; Step 2 asks for the PIN. The seal form only appears after both.

```
  {% elif o.status == 'assigned' and not (o.pickup_pin is none or (o.verified_at and o.pin_ok_at)) %}
    {% if o.runner_id == g.user.id %}
      {% if not o.verified_at %}
        <div class="card" data-wait-verify data-order="{{ o.id }}">
          <h2 style="margin-top:0">Step 1 of 2: show the shopper your phrase</h2>
          <p class="muted">Hold up your phone so the shopper can compare it with theirs. Don't read it out loud.</p>
          <div class="code phrase" data-phrase data-order="{{ o.id }}">{{ phrase }}</div>
          <p class="muted" data-phrase-left style="text-align:center"></p>
          <p class="muted">Waiting for the shopper to tap &ldquo;It matches&rdquo;...</p>
        </div>
      {% else %}
        <form class="card" method="post" action="{{ url_for('web.verify_pin', order_id=o.id) }}">
          <h2 style="margin-top:0">Step 2 of 2: enter the shopper's PIN</h2>
          {% if o.pin_locked %}
            <p class="badge bad">Locked after 3 wrong PINs. Call the admin.</p>
          {% else %}
            <p class="muted">The shopper has confirmed you. Ask them for their 4-digit PIN. {{ 3 - o.pin_fails }} tries left.<
            <input name="pin" inputmode="numeric" maxlength="4" pattern="\d{4}" autocomplete="off" required>
            <button class="btn">Check PIN</button>
          {% endif %}
        </form>
      {% endif %}
    {% else %}
```

9. Suggested talking points while recording

• Start with the problem: how does a shopper know the person is the real runner?

• Show **phrase_for**: HMAC of order secret + minute window, mapped into two word lists and a number. Same inputs on both phones, so the same phrase.

• Show **confirm_runner** then **enter_pin**: two independent proofs, 3-try lockout with `hmac.compare_digest` (constant-time compare).

• Show **record_pickup**: the server enforces the order of steps, not just the UI.

• Quick demo: two browsers (shopper and runner), phrase matches, tap, PIN, then the seal form unlocks.

10. Possible task ideas for the platform

• Add a unit test for **phrase_for** (same minute = same phrase, next minute = different, different orders differ).

• Add a test that **record_pickup** raises RuleError until **confirm_runner** and **enter_pin** succeed.

• Review/extend: lock-out reset, PIN length, phrase window length, or accessibility of the phrase screen.